3505. Minimum Operations to Make Elements Within K Subarrays Equal

Daily Question - November 4, 2026

Problem Statement

You are given an integer array nums and two integers x and k. You can perform the following operation any number of times, including zero: increase or decrease any element of nums by 1.

Return the minimum number of operations needed to have at least k non-overlapping subarrays of size exactly x in nums, where all elements within each subarray are equal.

Examples

Example 1: nums = [5,-2,1,3,7,3,6,4,-1], x = 3, k = 2. Output: 8. Explanation: Make [nums[1], nums[2], nums[3]] equal to 1 using 5 operations, and make [nums[5], nums[6], nums[7]] equal to 4 using 3 operations.

Example 2: nums = [9,-2,-2,-2,1,5], x = 2, k = 2. Output: 3. Explanation: Keep [-2,-2] unchanged and change [1,5] to [-2,-2] using 3 operations.

Constraints

2 <= nums.length <= 100000; -1000000 <= nums[i] <= 1000000; 2 <= x <= nums.length; 1 <= k <= 15; 2 <= k * x <= nums.length.

Solution: Sliding-Window Median + DP

For any fixed length-x window, the minimum operations needed to make all its values equal is obtained by changing every value to the median of that window. The cost is the sum of absolute differences from each window element to its median.

Compute cost[i] for every window starting at index i using a sliding-window median structure. Then use dynamic programming to select exactly k non-overlapping windows with minimum total cost.

Let dp[used][index] be the minimum cost after processing the first index positions and selecting used windows. From dp[used][index], either skip position index and go to dp[used][index + 1], or select the window starting at index and go to dp[used + 1][index + x] after adding cost[index]. The jump by x prevents overlap. The answer is the minimum dp[k][end] over all end positions.

Sliding-Window Median

Maintain the current window in two TreeMaps: lower contains the smaller half and upper contains the larger half. Keep lowerSize and upperSize balanced so that the largest key in lower is the median. Also maintain lowerSum and upperSum.

For a window with median m, cost = (m * lowerSize - lowerSum) + (upperSum - m * upperSize). This equals the total absolute distance from all elements to m and can be computed in O(1) after each slide.

Complete Java Solution

```
import java.util.*;

class Solution {
    public long minOperations(int[] nums, int x, int k) {
        int n = nums.length;
        long[] windowCost = getWindowCosts(nums, x);

        long[][] dp = new long[k + 1][n + 1];
        for (long[] row : dp) {
            Arrays.fill(row, Long.MAX_VALUE / 2);
        }
        dp[0][0] = 0;

        for (int index = 0; index < n; index++) {
            for (int used = 0; used <= k; used++) {
                if (dp[used][index] == Long.MAX_VALUE / 2) {
                    continue;
                }

                dp[used][index + 1] = Math.min(
                        dp[used][index + 1],
                        dp[used][index]
                );

                if (used < k && index + x <= n) {
                    dp[used + 1][index + x] = Math.min(
                            dp[used + 1][index + x],
                            dp[used][index] + windowCost[index]
                    );
                }
            }
        }

        long answer = Long.MAX_VALUE;
        for (int end = 0; end <= n; end++) {
            answer = Math.min(answer, dp[k][end]);
        }
        return answer;
    }

    private long[] getWindowCosts(int[] nums, int x) {
        int n = nums.length;
        long[] costs = new long[n - x + 1];

        TreeMap<Integer, Integer> lower = new TreeMap<>();
        TreeMap<Integer, Integer> upper = new TreeMap<>();
        long lowerSum = 0;
        long upperSum = 0;
        int lowerSize = 0;
        int upperSize = 0;

        for (int right = 0; right < n; right++) {
            int value = nums[right];

            if (lowerSize == 0 || value <= lower.lastKey()) {
                lower.merge(value, 1, Integer::sum);
                lowerSum += value;
                lowerSize++;
            } else {
                upper.merge(value, 1, Integer::sum);
                upperSum += value;
                upperSize++;
            }

            if (right >= x) {
                int outgoing = nums[right - x];

                if (lower.containsKey(outgoing)) {
                    removeOne(lower, outgoing);
                    lowerSum -= outgoing;
                    lowerSize--;
                } else {
                    removeOne(upper, outgoing);
                    upperSum -= outgoing;
                    upperSize--;
                }
```

```
            }

            if (lowerSize < upperSize) {
                int moved = upper.firstKey();
                removeOne(upper, moved);
                upperSum -= moved;
                upperSize--;

                lower.merge(moved, 1, Integer::sum);
                lowerSum += moved;
                lowerSize++;
            } else if (lowerSize - upperSize > 1) {
                int moved = lower.lastKey();
                removeOne(lower, moved);
                lowerSum -= moved;
                lowerSize--;

                upper.merge(moved, 1, Integer::sum);
                upperSum += moved;
                upperSize++;
            }

            if (right >= x - 1) {
                long median = lower.lastKey();
                costs[right - x + 1] =
                        median * lowerSize - lowerSum
                        + upperSum - median * upperSize;
            }
        }

        return costs;
    }

    private void removeOne(TreeMap<Integer, Integer> map, int value) {
        map.merge(value, -1, Integer::sum);
        if (map.get(value) == 0) {
            map.remove(value);
        }
    }
}
```

Correctness

For a fixed multiset of x values, the median minimizes the sum of absolute differences, so it minimizes the number of +1 and -1 operations required to equalize that window. The sliding structure correctly computes this cost for every length-x window.

For the selection problem, any valid solution consists of k disjoint windows. The DP transition either skips one position or consumes exactly x consecutive positions, so selected windows cannot overlap. Conversely, every sequence of k non-overlapping windows corresponds to a path in the DP. Therefore, the minimum dp value after selecting k windows is the required answer.

Complexity

Time: O(n log x + n k). The first term comes from inserting and removing each element from the two TreeMaps; the second comes from the DP transitions. Auxiliary space: O(n + n k) for the window-cost array and DP table.

Implementation Notes

Use long for costs, sums, and DP values because operations can be as large as roughly 10^6 per element and there can be up to 10^5 elements. Initialize unreachable DP states with Long.MAX_VALUE / 2 to avoid overflow when adding a window cost. The final answer is long, matching the expected return type.